

PLUG & CHUG

理解 AI Agent， 搭建自己的工作系统

已经有 Codex、Claude Code、WorkBuddy 这些好用的应用，为什么还需要自己的本地 Agent 系统？



@kwis7

Reader edition 3 / 2026-09-22

github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide

阅读路线

让你正在使用的 AI 带着搭建；用本书理解每一步，需要练手时再使用贯穿全书的案例。

搭建一个下次还能接着用的 AI 工作空间	3
开始之前：Agent 到底怎样工作	4
1. 先完成一项小任务	9
2. 给工作一个可以继续的位置	11
3. 换一个会话，继续刚才的工作	12
4. 把有效的方法留下来	13
5. 职责开始冲突时，再拆分工作	15
6. 把工作带到另一个工具中	16
7. 分清到底完成了哪一步	17
8. 维护真正用得上的积累	19
9. 参考：使用标准工具包	20
延伸阅读	22

搭建一个下次还能接着用的 AI 工作空间

开始这份教程，不需要先学编程。先从你熟悉的工作出发：阅读、写作、学习、研究，或者一件希望 AI 帮忙的日常事务。目标是搭出一套你能看懂、明天还能继续使用的 AI 工作系统。

聊天结束后，好工作应该留下来。来源、决定、有效方法、检查过的成果和未完成任务，需要放在下一次会话能找到的地方。你已经在用的 AI 应用，可以协助创建这个地方，并在过程中解释每一部分。

从你自己的 AI 助手开始

打开[新手指南](#)，把其中的起步提示词交给平时使用的 AI 应用。仓库链接已经包含在内。助手先问少量问题，了解你的工作、软件和电脑，再提出适合你的起步方案。你看过文件准备放在哪里、确认后，它再处理有能力执行的技术步骤。

第一个有用的成果很具体：一个本地工作空间、一份核对过的结果、一段交接说明，以及一张“下次打开什么、说什么”的使用卡。再开一个会话，试着按这张卡继续。设计方案、创建文件夹、成功恢复工作是不同的进展，助手应说明哪些已经实际完成。

本书解释这条路线中的选择。不懂某个术语时，可以让助手结合你的任务解释。终端命令是可选的实现路线，集中在最后的工具参考章和[快速开始](#)中。

模型可以更换，工作可以保留

当前使用的模型可以变化，已经积累的工作仍然属于你。来源、决定、方法、任务记录和检查过的结果，应该以你能查看、能带走的形式保存。换一个界面或模型服务，可能需要调整接入方式，但不必因此从头积累。

我们把围绕模型执行配置好的工作环境称为 **Harness**，包括指令、文件、工具、任务状态和检查。其中一部分是普通文字，另一部分是授予访问能力或检查操作的软件。后文会逐步区分它们。

公司的比喻可以帮助理解：模型像当前负责推理的员工，Harness 保存工作安排、记录和工具，你是所有者。这个比喻说明的是职责，不代表模型具有人的忠诚、意识，或自动访问所有文件的能力。

借一个例子学做法，再用到自己的工作中

第 1-4 章用三个虚构场地的比较贯穿讲解。现成笔记让你不用提供个人信息，就能看清如何检查结果、保存成果、找回下一步。可以跟着做这项可选练习，也可以把相同做法用在引导搭建时选定的任务上。搭建个人系统，并不要求先拥有一个“场地比较 Agent”。

第 5-8 章说明何时需要更多职责、其他工具、更严格的检查和后续维护。最后的参考章连接这些思路与标准工具包。每一部分都应该回应工作增长后实际遇到的问题。

教学工作空间有意保持简单。标准生成器则提供更完整的结构，包括任务记录和本地检查。助手应解释哪一种适合当前需求；更小的教学工作空间，不是生成器里未写明的选项。

让结论与证据对应

练习中的资料是合成数据。仓库命令与兼容性记录说明已有实现，不代表你电脑上安装的应用已经通过验证。文字指令、静态检查、新会话行为以及外部交付，各自需要对应证据。

接下来可以读 [Agent 到底怎样工作](#)，借公司图理解基本概念。阅读后面的例子时，始终带着自己的任务想一想：什么会让下一次接着做更容易？

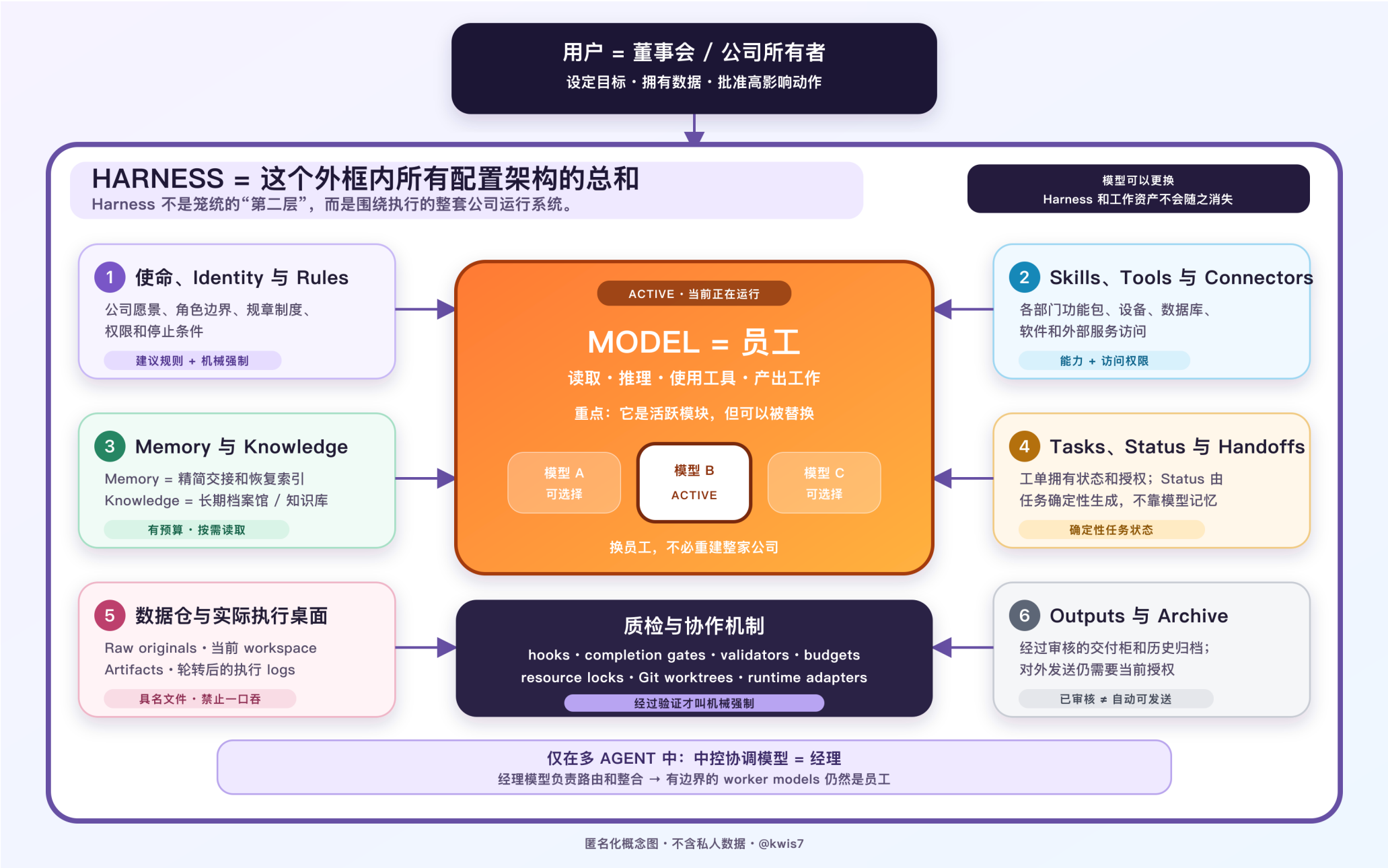
开始之前：Agent 到底怎样工作

你请 AI 比较三份资料。它读取文件、写出表格、检查一个缺失的信息，又留下笔记供下次使用。到底是什么在完成工作：模型、Agent、提示词，还是 Skill？

它们承担的作用不同。分清这些作用，你就更容易搭建一个真正有用的系统，也不会把文件夹误认为能力，或把一段好提示词当成可靠性的保证。本书所说的 **Agent**，是通过模型、指令、工作状态、可用工具和执行循环来承担某种职责的系统。具体实现可以不同：Agent 能在单次会话中运行，不一定需要本地文件，也不必有其他 Agent 配合。

把图看作一家正在工作的公司

下面这张图展示了本仓库的完整设计。它帮助你看清各部分职责，不要求你在开始之前就建好每一个方框。



当前模型位于用户配置的完整 Harness 中。

最上方，**你是所有者**：决定目标，掌握自己的材料，并授权具有重要后果的动作。中间，**模型是当前工作的员工**：理解收到的材料，对请求进行推理，再提出回答或工具调用。模型不是整家公司，也不是公司的档案和权限系统。

Agent 是一种工作职责，例如负责比较来源的研究助手。指令说明任务与边界，状态记录进展，工具提供可执行的操作。同一个模型可以在不同会话中承担多个角色，使用不同的上下文和权限；同一种职责也可以先后由不同模型承担。给它取名“研究 Agent”，并不会创建一个单独训练的新模型。

周围的 **Harness** 是支持执行的整套配置：规则、文件、方法、工具、状态和检查。在图中，这些安排与当前模型处在同一个系统边界里。更换模型时，已有工作可以保留；如果同时更换运行环境，适配方式和验证也可能需要调整。

公司的比喻用来区分责任：Skill 像操作手册，工具像设备，记忆像简短交接，知识像参考资料库。只有多个角色需要路由和整合时，协调模型才承担经理职责。它仍然只是一个有明确边界的模型角色，不是必需的额外层级，也不是更高的授权来源。

这个比喻不代表意识、忠诚、永久记忆或自动访问能力。图中的质量控制，只有在对应软件已经实现并验证时才实际生效。写下规则，与机械地阻止某个动作，是两回事。

先分清几个基本概念

概念	在系统中承担什么作用
模型（model）	理解收到的上下文，生成回答或提出动作
Agent	通过指令、状态、工具和执行循环承担一项职责
提示词（prompt）	向模型传达请求、指令或其他材料
技能（Skill）	打包可复用的流程与配套资源
工具（tool）	完成读文件、执行计算等具体操作
运行环境（runtime）	承载执行、提供上下文、调用工具并处理返回结果
Harness	配置模型执行所需的工作环境
文件（file）	保存能够跨会话留存的信息
上下文（context）	模型在生成当前回复时实际获得的材料

这些是设计中的分工，不一定分别对应不同产品。一个应用可以同时提供执行环境、工具、存储和界面。固定 workflow 按预先设定的顺序运行；在 Agent 的执行循环里，模型可以根据观察结果，在系统限制内决定后续步骤。实际系统可以结合两者。这一区分参考了 [Anthropic 对工作流与 Agent 的解释](#)，不代表对某个具体产品实现的认证。

提示词交代任务，Skill 提供方法

一次任务的提示词可以是：“根据给定笔记，比较适合 16 人在 18:00 之后使用、且必须有无台阶入口的场地。”它说明现在要做什么。长期规则则表达在一类工作中应持续遵循的要求，例如保留来源原文，或者让没有依据的信息保持未知。两者在被加载后都能影响模型，但适用范围和保存期限不同。

模型收到的输入不局限于你最新发出的聊天消息。运行环境也可以把相关指令、已加载的 Skill 内容和工具返回的观察结果，组装进当前上下文。

在本仓库采用的形式中，**Skill 的核心就是一份保存在 SKILL.md 里的可复用指令文档**。加载以后，其中的文字成为模型当前上下文的一部分。你可以把它理解为：把一份好用的提示词或方法“置顶”（pin）在固定、容易发现的位置，以便以后可靠地找到并重复使用。

这里的“置顶”说的是容易找回，不是永久激活。[Agent Skills 规范](#)采用先通过名称和描述发现技能、需要时再加载完整说明的方式。这不表示每轮都把全文塞进上下文，也不会自动变成更高优先级的系统消息。实际发现和加载仍取决于宿主支持。

文档可以教模型逐项检查要求、引用来源，也可以附带示例、参考资料或脚本。这些材料提供方法，不是在训练新的神经能力或修改模型权重，更不会授予工具权限。附带脚本仍需要实际执行工具按访问规则调用；写到邮件，不代表已经连接账户或获准发送。

因此，同一个模型同时收到任务、来源笔记和比较流程时，会比只收到“帮我选一个场地”有更清楚的判断依据。更好的上下文可以引导行为，但结果仍然需要检查。

沿着一次执行过程看清分工

首次练习使用合成笔记，并非真实场地信息。请求是比较，不是预订。一次典型的工具使用过程如下：

1. **接收任务。** 运行环境提供请求和相关指令。模型识别要求，判断需要哪些来源文件。
2. **判断下一步。** 如果还看不到笔记，模型应提出读取文件，而不是把文件名当成证据。
3. **请求工具。** 调用请求说明操作名称与参数。回复一句“我读过文件了”，并不等于真的执行了读取。
4. **由运行环境执行。** 宿主依据实际访问策略调用获准操作，可能返回文件内容，也可能拒绝访问或报告错误。
5. **使用返回结果。** 工具返回的材料进入后续工作上下文。模型比较笔记，必要时继续请求获准的证据。
6. **复核并保存。** 草稿写入 workspace/comparison.md。依据来源复核、纠正之后，检查过的版本进入 outputs/comparison.md，实际进展记录在 workspace/handoff.md。
7. **结束或交接。** 说明已经完成的结果与剩余限制。信息或授权不足时，下一步可能是暂停，而不是继续操作。

Cedar Hall 的给定笔记支持已记录的要求；Willow Room 的开放时间不合适；Maple Studio 没有说明无台阶入口。这个过程应当保留未知项，因为推理不能补出来源中不存在的入口说明。起草一条尚未发送的问题，可以是下一项任务，但写出问题不等于获得答案。

每一步都可能出错：路径写错、工具不可用、权限被拒绝、返回内容不完整，或模型理解有误。文件写入成功，也不能证明内容正确。没有文件工具时，聊天界面可以返回文字，由你保存，但不能声称自动写回已经发生。第 1 章会把这个过程变成一次具体练习。

Markdown 文件到底做了什么

.md 是 **Markdown** 常用的文件扩展名。Markdown 是一种可编辑的纯文本格式：标题可以以 # 开头，列表可以用短横线，链接可以指向其他文件。不借助特殊软件，也能直接阅读其中的文字。它适合保存指令、笔记和交接，因为内容容易检查、修改和带走。

某些文件名只有在宿主约定下才有特殊意义。当运行环境能够在预期位置发现并加载它们时，AGENTS.md 或 SKILL.md 才能作为指令或技能入口。它们不是到处都能执行的文件。RULES.md、workspace/handoff.md 等名称则表达项目内的约定；工作流程需要安排读取相关内容。

下面几段小示例展示不同文件的职责。它们不是首次练习必须额外创建的文件，也不是一项可直接安装的完整 Skill：

```
task-brief.md
比较适合 16 人在 18:00 之后使用的给定场地。
必须有无台阶入口。

RULES.md（示意的项目规则）
来源缺失的事实保持未知，不要编造。

skills/source-comparison/SKILL.md（示意流程）
依据每份来源逐项检查要求，并注明证据。

workspace/handoff.md（仅在实际发生后记录的示例状态）
草稿已保存到 workspace/comparison.md；尚待来源复核。
```

这里有三种不同的东西：**已保存的文件、当前上下文和模型权重**。文件保存文字；上下文包含为本轮工作选入的指令、对话和工具结果；权重则是模型通过训练获得的参数。保存或读取这些文件，本身不会更新模型参数。新会话靠检索相关记录恢复工作，并不是因为每条保存的事实都已经进入模型内部。

通过明确反馈，让系统适合自己

个人系统应当服务于你真正反复进行的工作。你可以明确指定语言、详细程度、证据标准、复核时点和交付形式。这些选择在范围清楚、能够修订时，才会持续有用。

假设你纠正一份比较：“以后的来源比较，先给简短建议，再解释依据与未知项。”这项明确偏好可以成为比较工作的规则，不必自动适用于诗歌、调试日志或所有未来对话。记录你提出的要求，不要推断你具有某种人格。

其他经验应放在不同位置。“Maple 的入口仍然未知”属于当前任务状态；反复使用的提取、比较、引用和复核步骤，可以成为 Skill；为什么来源未说明不等于否定答案，可以写成知识笔记。这样就不会让一个巨大的记忆文件混装偏好、未完成任务、流程和参考资料。

个性化改变的是模型可获得的信息与工作方法，不是自动训练。反复使用，也不能证明一个猜测的偏好正确。不要从普通纠正中推断敏感特征；只保留适当且获准的材料，并复查旧偏好是否仍然适用。

已经有 AI 软件，为什么还要搭建自己的系统

如果你已经使用 Codex、Claude Code 或 WorkBuddy，这里要做的是在所选软件中组织好自己的工作。本手册不假定这些产品缺少记忆、Skill 或工作空间功能。优先复用所选产品已有的运行环境、工具、指令加载与存储能力，无需另写一套调度引擎或定制应用。

个人系统是你有意组合起来的工作安排：稳定职责、自己的获准来源、有明确范围的偏好、反复有效的方法，以及带证据的当前状态。软件提供能力，你决定它们如何服务于自己的工作。有些人直接使用产品的内置安排，就已经足够。

这里的**本地**，是指把长期工作空间保留在自己能够掌握的位置。文件可以查看、备份，并在适当情况下进行版本管理，追踪哪些内容发生了变化。来源与结论保持联系，新会话可以找到下一步，其他兼容工具也可以复用获准材料。

文件保存在本地，不代表模型也在本地运行，更不表示整个流程离线。使用托管模型或连接器时，内容仍可能被发送出去。隐私取决于实际数据流、访问权限、服务设置和你提供的材料，不能由 .md 扩展名或文件夹名称保证。迁移时也要检查新工具如何加载指令、保存结果；工具之间并非通用即插即用。

当工作重复出现、跨越会话，或需要可追溯的结果时，这种安排会有价值。一次性的解释，普通聊天也许已经足够。长期系统也有维护成本：指令会过期，副本会分歧，方法需要复查。先完成一件有用的小事，只保留能让下次更容易的部分。[首次任务章节](#)会带你迈出这一步。

1. 先完成一项小任务

这一章借可选的虚构练习，把做法展示出来。如果已经在[新手指南](#)中选好了自己的任务，可以把相同顺序用在那里：约定结果、使用具名资料、检查、保存，再留下交接。

假设你正在准备一场 16 人参加的工作坊，需要在 18:00 之后使用场地，而且所有人都必须能够通过无台阶入口进入。在联系任何场地之前，你希望先有一份简短比较，分清哪些选项满足条件、哪些不满足、哪些还无法判断。

[首次项目练习](#)提供三份合成的来源笔记。它们代表虚构场地，并非真实商家或经过实时核实的信息。这是一项资料比较练习，不包含预订、发消息或付款的授权。

先看什么样的结果有用

一份有用的答案不只是排名。它应当展示资料记载的事实，用同一套条件检查每个选项，说明判断如何得出，并保留未知项。

场地	容量	记载的开放时间	无台阶入口	判断
Cedar Hall	18 人	18:00–21:00	有	满足已记录的条件
Willow Room	24 人	09:00–17:00	有	不满足晚间使用要求
Maple Studio	20 人	18:00–22:00	未说明	无法确认无障碍条件

根据这些资料，Cedar Hall 是满足条件的候选项。这个结论有清楚的范围：它不能证明某个日期有空位，也没有确定活动时长、价格或预订条款。Maple Studio 没写入口情况，表示我们不知道，不能据此断言它没有无台阶入口。Willow Room 容量更大，也不能弥补开放时间不合适的问题。

使用你已有的工具

选择你已经能用的 AI 工具。如果希望它直接处理文件，它需要能读取练习资料，并且有权写入你选定的工作副本。开始前确认实际权限。如果界面只能接收粘贴文本或上传文件，就明确提供三份笔记，再自行保存它返回的 Markdown。这仍然能练习方法，只是文件加载和写回由你完成。模型或 API 的正常使用可能产生相应费用；练习本身不需要付费数据服务。

如果想保留仓库中的参考文件，先把练习复制到自己的本地练习目录，并告诉 Agent 它可以修改哪一份。从练习目录出发，可以这样交代：

```
请读取 task-brief.md、sources/cedar.md、sources/willow.md
和 sources/maple.md。暂不读取 expected/。
只依据这些给定资料，比较适合 16 人在 18:00 之后使用、
且必须有无台阶入口的虚构场地。
将来源链接、判断和未知项写入 workspace/comparison.md。
输入缺失时明确指出，不要猜测。
不要联网、联系或预订。覆盖已有结果前，先商定具体改动。
```

教程中的 Agent 首先创建草稿。如果使用纯聊天界面，由你完成保存，并将本地文件加载标为未测试。仓库里的 expected/comparison.md 和 expected/handoff.md 是参考答案，不能证明你自己的练习已经完成。先尝试起草，再打开它们。

接受结果之前，沿着来源检查

打开自己生成的 workspace/comparison.md，沿着链接查看 sources/ 下的三份笔记，亲自核对容量与时间。再检查推理：三个场地容量都足够，两个有晚间开放时间，而只有 Cedar 同时有晚间时间和明确记载的无台阶入口。现在可以与 expected/comparison.md 对照。

如果答案说 Maple 也符合要求，就追问哪份资料证实了入口情况。正确的修正是把这一项标为未知，修改判断，并留下纠正记录。再流畅的文字也不能补出资料里不存在的字段。

复核并完成必要修正后，请 Agent 把检查过的版本保存到 outputs/comparison.md，保留原草稿。再让它创建 workspace/handoff.md，记录实际检查、剩余未知、允许读写的路径，以及下一步：起草一条询问

Maple 入口情况但不发送的问题。真正观察新会话接手之前，把恢复测试记为未测试。真实预订仍然需要新信息与单独授权。

2. 给工作一个可以继续的位置

完成的比较保存了一个答案。一个有用的工作空间还会保存理解答案所需的东西：输入、要求、检查，以及尚未完成的事项。缺少这些内容，下一次会话很容易重复调查，或者沿用已经忘记适用范围的结论。

我们的例子只需要一个小目录。给定笔记与工作结果、交接记录分开放置，仓库自带的 `expected/` 保留为教学参考。这样的安排不需要新建 Agent、数据库，也不必先生成完整项目骨架。

让每个文件有明确用途

来源笔记说明提供了什么资料。`workspace/comparison.md` 是草稿；复核后，`outputs/comparison.md` 成为检查过的答案。`workspace/handoff.md` 记录当前状态和下一项允许执行的动作。这些文件各有用途，草稿并不是最终结果的第二个权威版本。如果决定变化，先复核修改，再更新结果，并在交接记录中写清状态。

工作增加以后，下面几种区别会越来越有用：

信息类型	回答的问题	例子
来源资料	提供了什么证据？	Cedar Hall 记载的开放时间
任务状态（task state）	现在做到哪里？	比较已检查，尚未联系场地
记忆（memory）	哪条简短线索有助于恢复工作？	当前任务位置与关键决定
知识（knowledge）	哪种稳定认识值得复用？	证据缺失如何影响比较判断
技能（skill）	哪种做法应该重复使用？	逐一检查候选项是否满足每项要求

上下文（context） 则是另一个概念：它指当前会话实际交给模型的材料。文件存在于磁盘上，不代表已经进入上下文。“请读取来源”这条指令，也不同于来源已经被读取的证据。需要确认加载情况时，让 Agent 指出支持答案的文件及其中的事实。

保留来源，也保留未知

不要为了让比较更容易，把 Maple 的来源笔记改写成一个确定答案。输入中保留“入口情况未说明”，在结果中解释这会如何影响判断。如果后来取得新证据，记录它的来源和日期，再更新受影响的判断。这样不必保存整段聊天，也能追溯结论为什么变化。

真实工作同样如此。摘要是对来源的解释，不能取代来源信息。分开记录资料的说法与自己的结论，保留足够的出处，让复核者能检查两者之间的推理。只保存你有权保留、并且有权交给工具使用的材料。

有需要时，再增加结构

标准项目骨架为较大的工作提供了约定位置：`raw_data/` 保存原始输入，`workspace/` 放草稿，`artifacts/` 放生成的中间产物，`logs/` 记录执行轨迹，`outputs/` 放经过检查的结果，`archive/` 保存已结束的材料。完整职责见[文件系统契约](#)。

这些名称表达用途，但不会执行权限控制。一个能读取整个目录的工具，仍然可能接触名叫 `private` 的文件。因此，敏感工作需要实际的访问限制，并明确决定哪些资料可以进入模型。公开示例应当使用合成或获准公开的内容，不能只是把私人记录换掉名字。

增加文件夹之前，先做一个简单测试：能否在一分钟内找出当前答案、它的来源和下一步？如果可以，现有结构也许已经足够。如果不行，就解决造成耽搁的具体歧义。每一层结构，都应当降低继续工作的成本。

3. 换一个会话，继续刚才的工作

工作能否延续，是可以测试的。结束原来的对话，打开新会话，看看保存的文件是否足以支持下一步。这比询问原来的模型“下次还记得吗”更有价值。

在场地练习里，比较本身已经有用。交接记录要防止后来的会话把它误认为已经完成的预订，也要防止 Maple 的未知项悄悄变成“符合条件”。

留下能指向下一步的交接记录

交接记录是一份简短工作说明，不是聊天全文。它应当让接手者知道原任务是什么、已有文件在哪里、结论为什么成立，以及什么新信息会改变结论。例如：

任务：比较三个虚构场地，要求容纳 16 人、可在 18:00 之后使用，
并无台阶入口。
结果：`outputs/comparison.md`。草稿：`workspace/comparison.md`。
来源：`sources/cedar.md`、
`sources/willow.md`、`sources/maple.md`。
已检查：容量、开放时间，以及来源中的入口说明。
判断：Cedar 满足已记录条件；Willow 的时间不合适；
Maple 的入口情况仍然未知。
限制：仅使用合成笔记。空位、价格、准确活动时长尚未核实。
未获得联系或预订授权。
下一步：确认复核结果存在，再起草一条询问 Maple 无台阶入口的
问题，保存到 `workspace/maple-question.md`，但不要发送。
只读本交接记录、`task-brief.md`、复核结果和三份具名来源；
只写这条问题与本交接记录。
恢复测试：观察新会话实际接手前，保持未测试。

这段记录保留了决定和边界，无需保留所有起草过程。[expected/handoff.md](#) 提供了可供对照的参考，但你仍应当如实记录自己这一轮实际做过的检查。

做一次重新接手练习

在练习副本中打开新会话，或者明确向它提供交接记录和具名来源。请它先读 `workspace/handoff.md`，只检查获准读取的文件，说明当前结论与未解决问题。然后执行记录中的下一步：起草一条询问 Maple 无台阶入口的问题，保存到 `workspace/maple-question.md`，并更新交接记录。不得联网、联系或预订。

成功接手意味着：找到复核结果，识别出 Cedar 是证据支持的候选项，保留 Maple 的未知项，并理解 Willow 的时间问题。重新打开新问题和更新后的交接记录，再记下测试工具、日期与实际结果。写出问题不等于取得答案。具名输入缺失时，应报告缺口，不能凭记忆重建证据。

还可以在另一个新会话中，按照教程的缺失来源提示练习失败处理。只允许读取 `task-brief.md`、`sources/cedar.md` 和 `sources/willow.md`，声明 Maple 笔记不可用，但不要删除原件。禁止读取其他练习文件或使用记忆中的事实。把有限比较写入 `workspace/missing-source-check.md`，让 Maple 保持未评估，并保留原复核结果。这里测试的是声明的输入边界；机械隔离仍取决于工具权限。

重试之前，先确认留下了什么

工作中断时，常常已经留下部分结果。重试前检查草稿、复核结果和 `workspace/handoff.md`，找出最后成功完成的一步。复核结果缺失时应回到复核步骤，部分草稿不能当成检查过的输出。重新读取资料，或重新生成可替换的草稿，通常容易处理。重复执行外部动作，则可能造成重复预订、重复消息或重复付款。

外部操作结果不明确时，先查外部记录。超时说明没有收到响应，并不证明操作没有发生。虚构练习没有外部动作，但先练习这种区分，有助于以后处理更复杂的流程。

较大的工作可以使用工具包的 `tasks/<task-id>/task.yaml` 保存状态，并从中自动生成 `STATUS.md`。它增加了明确的范围、验证和生命周期字段，但基本习惯没有变化：当前事实只记录一次，保存有用的部分成果，把下一步写得足够具体，让新会话能够接着做。

4. 把有效的方法留下来

做过几次比较后，你可能发现，真正值得保留的是流程：先明确要求，再从每份来源提取相同字段，逐项判断，并展示证据缺口。场地、文档和当前模型都可以变化，这套方法仍然有用。

Skill 的核心是一份 Markdown 指令文档，通常命名为 `SKILL.md`。宿主加载它以后，其中的文字就成为模型上下文的一部分。你把好用的提示词或方法放到一个容易发现的位置，就像把操作手册“置顶”（pin），方便以后重复使用。模型依靠已有能力遵循这份说明；保存 Skill 本身不会训练新的模型权重。

这里的“置顶”是为了找回方法，不代表每轮都加载全文，也不自动提高指令优先级。名称和描述帮助宿主选择，完整流程与参考资料按需加载。Skill 还可以附带示例、模板或脚本，但脚本仍然需要实际工具和执行权限。当重复工作或反复出现的错误值得专门处理时，维护一项 Skill 才有意义。

提炼方法，留下具体案例的边界

“选择 Cedar Hall”是一次虚构练习的结论；“检查每个候选项是否满足所有必要条件”才是可复用的方法。把两者混在一起，下一次比较就可能继承旧答案。

针对这项练习，可以这样描述一个有用的 Skill：

名称：有来源支持的比较
适用情况：根据明确要求，比较用户提供的候选项。
输入：要求与具名来源笔记。
方法：提取事实并标注来源；逐项检查要求；
区分满足、不满足和未知；解释最终判断。
输出：简短比较、待解决问题和交接记录。
限制：不推测缺失事实，不执行外部动作。

这只是教学示例，不是可在所有工具中直接安装的通用格式。仓库的 [Skill 模板](#) 展示了打包形式。能否被发现和加载，取决于所选运行环境与安装位置。

让触发条件真正帮助选择

描述应当让系统在加载完整内容之前，就知道何时使用这项方法。“帮助研究”太宽泛，无法说明它与摘要、写作、数据收集有何区别。有效的描述会写清工作类型、输入、输出与不适用情况。

用几条普通请求试一试。“根据我们的要求比较这三份供应商资料”应当选中比较方法；“写一封感谢信”不应当选中。“比较这些场地并预订最合适的一个”包含两个不同动作：Skill 可以完成比较，而预订需要单独的能力和授权。触发一个流程不能悄悄扩大权限。

然后检查方法本身。场地练习应当得到三个不同判断：Cedar 有证据支持，Willow 时间不符，Maple 的无障碍条件待确认。再加入缺失来源或信息冲突的变化，看看流程是否正确处理不确定性，而不是只会复述参考答案。

把经验放到合适的位置

始终适用的偏好，应当进入相关工作规则。稳定背景进入知识。今天尚未完成的比较进入任务状态。反复执行的步骤进入 Skill。真正需要执行或访问外部服务的能力，由工具承担。

这样分开以后，Skill 才容易理解和维护。短描述用于发现，主体说明方法，详细参考只在任务需要时加载。每轮对话都加载所有 Skill 和参考材料，会消耗注意力，也破坏这种按需使用的安排。

共享做法，同时保留数据归属

另一个 Agent 可以借用比较方法，而不必收到私人候选资料、内部决定或完整聊天记录。分享流程和合成示例，再由接收方提供自己的获准输入。方法因此能够传播，数据归属也仍然清楚。

新 Skill 应在实际使用后复查。针对可识别、可复现的问题保留修正，不必把每个特殊请求都变成永久规则。更详细的[方法提炼指南](#)说明了如何保留重复经验，同时避免积累大量重叠流程。

5. 职责开始冲突时，再拆分工作

一个 Agent 配合几项合适的 Skill，已经可以承担相当多的工作。出现新主题或新的写作格式，并不自动意味着需要另一个 Agent。当反复进行的工作具有不同使命、数据边界、来源体系、权限或交付周期时，拆分才开始有价值。

先想清楚要避免哪种问题。如果研究结论还在变化，设计者却同时修改最终报告，明确划分职责可能有帮助。如果问题只是表格格式不一致，模板或 Skill 往往已经足够。

选择合适的组织单位

实际需要	通常可以从这里开始
一个临时目标	任务或项目
跨任务重复的方法	Skill
稳定的参考材料	知识集合
有独立归属的长期工作	领域 Agent
一项边界清楚的独立贡献	临时子 Agent (subagent)
访问服务或执行具体动作	工具或连接器 (connector)

领域 Agent 表示一种持续的职责，通过指令、工作文件和访问安排体现出来。它不必永久绑定某个模型。换一个模型，改变的是当前推理组件，不应悄悄改变资料归谁负责，或者哪些动作获得了授权。

场地练习只需要一个 Agent。更大的长期活动工作流，可能由研究方维护来源比较，再由沟通方准备获得批准的消息。调查选项与联系场地有不同的权限要求。职责分开后，边界更容易检查，但实际工具权限与明确的行动规则仍然必要。

委派之前，先写清责任

一项清楚的委派，应说明需要什么结果、能读哪些输入、可以在哪里写入、哪些动作禁止，以及如何检查产出。只有“研究一下”四个字，会留下太多不同理解的空间。

例如，可以请一名临时复核者读取三份虚构笔记，指出 workspace/comparison.md 中缺乏支持的判断。它需要读取这些文件，并有一个提交复核意见的位置；无需改写来源、改变任务范围或联系场地。主任务仍然负责整合意见、检查最终答案，再保存到 outputs/comparison.md。

当一个看似合理的错误可能逃过自查时，独立复核尤其有用。但它也增加时间与上下文成本，应当有具体理由。第二个 Agent 如果只重复相同假设、没有查来源，并不会显著增加可信度。

同一权威文件只安排一个写入者

两个 Agent 可以读取同一来源。两个 Agent 同时修改同一份当前状态，却可能互相覆盖。给每条权威路径安排一个写入者，其他参与者另写建议或产物。

Git worktree 可以把并行代码修改放在不同检出目录中。资源锁（resource lock）用于协调共享文件、设备、端口等资源的占用。它们处理的问题不同：worktree 不会隔离共享账户，锁也不是隐私保护。两者都不会授予发布或执行外部动作的权限。

当多个长期负责人需要路由或协调时，中控才开始有用。它负责知道工作归谁、管理依赖、整合结果，但不会因此自动拥有每个领域的私密事实。不同 Agent 可以共享方法，数据则只在获得授权的范围内传递。

用普通请求测试职责边界

为每个 Agent 写两条明确属于它的请求、两条不属于它的请求，再写一条与相邻职责重叠的请求，检查描述能否合理分配工作。[路由评估指南](#)提供了更完整的方法。

如果无法说明新增 Agent 为什么需要独立责任，就先把任务留给现有负责人。等拆分能让一个真实边界更清楚时再增加结构，并始终为每份最终结果保留一个明确的主负责人。

6. 把工作带到另一个工具中

可迁移的工作，应当在原对话之外仍然容易理解。场地笔记、比较、交接记录和可复用方法，都能由人阅读，也能提供给另一个模型。这是可迁移性的基础。自动加载、工具访问和规则执行，则需要额外适配。

更换模型与更换运行环境是不同的操作。**模型（model）**负责推理；**服务提供方（provider）**通过服务或后端提供推理能力；**运行环境（runtime）**承载会话、组装上下文、提供工具，并可能管理权限或钩子（hook）。**切换层（switchboard）**可以管理模型或服务配置，却不因此拥有任务记录和工作文件。

先确定哪些东西需要带走

从最小但有用的一包材料开始：任务要求、获准使用的来源、当前结果、交接记录，以及相关方法。再加入影响这项工作的规则。目的工具支持上传大量文件，不代表应该上传整个私人工作空间。

接着确认目的工具如何接收这些内容。本地工具可能识别指定入口文件；托管工作空间可能要求填写项目指令并上传选定文件；自行开发的 API 应用则需要自己组装上下文、实现保存逻辑。一份写着“请读取我的文件”的文档，不能让本来没有文件访问能力的界面突然获得这种能力。

还要决定修改如何回到权威工作副本。如果新工具只在聊天中给出修改后的比较，就需要有人保存回去。没有对齐规则的两份副本，即使各自模型推理正确，也会逐渐分歧。人工写回完全可以是一种有效流程，前提是分工明确。

把兼容性记录当作证据说明

仓库的[兼容性清单](#)记载了产品类别、生成的配置、限制和证据状态。其记录的审查日期为 2026-08-09。在这份记录中，Codex、Claude Code 和 Gemini CLI 被标为 `verified_static`，新会话验证为 `not_run`。这描述的是仓库保存的证据，并不表示本手册重新认证了厂商当前的产品行为。

其他条目还区分已文档化的接入、人工映射（manual projection）、暂定支持、模型服务、切换层和自建 Harness 模式。适配文档有参考价值，但文件存在不能证明产品已经加载它，或已经执行其中的规则。先读[适配选择指南](#)，再读目标工具的适配文件。安装或改变接入方式时，还应重新核对该产品的官方说明。

用小练习检查目的环境

迁移重要工作前，先用合成的场地练习。在目的环境中新开会话，分别检查四件事：

1. 能否识别预期规则和具名来源文件。
2. 能否正确完成比较，并保留 Maple 的入口未知项。
3. 能否把结果与交接保存到约定位置，或者明确说明保存由你完成。
4. 如果声称存在机械完成检查，能否拦下故意不完整的任务，并放行有效完成的任务。

记录工具版本、日期、测试材料、观察结果和剩余缺口。模型说“我理解规则”，不能证明完成关卡真的生效。另一方面，没有关卡不妨碍人工复核一份比较；它改变的是你能声明到哪一步，以及哪些检查需要自己承担。

在新路线通过检查前，保留原工作副本。迁移的目标，是带走已有证据和方法，同时调整依赖具体运行环境的部分。长期材料和迁移限制都清楚，可迁移性才真正可用。

7. 分清到底完成了哪一步

“完成”可能表示文件已经生成、比较已经复核、测试已经通过，也可能表示某个外部动作已经成功。这些结论需要不同证据。可靠的工作方式，会在开始前说明期望结果，再检查与该结果对应的事实。

对于场地练习，完成意味着比较符合给定笔记、判断遵循明确要求、未知项仍然可见，并且留有可用的交接记录。它不意味着工作坊已经有了确认预订的场地。最终回复和下一次会话都应保留这个区别。

让检查对应声明

想要声明的结果	应当检查的证据
结果已保存	打开准确的文件并查看内容
比较有证据支持	把关键事实与判断逐一追溯到来源
脚本检查通过	记录命令、退出状态和相关输出
文档可以使用	渲染、检查页面，并测试导航
运行环境加载了指令	观察新会话是否使用预期入口
完成关卡生效	在实际环境中观察无效与有效案例
外部动作成功	核对服务的权威结果或回读记录

验证记录 (receipt) 保存实际发生的事情：做了什么检查、何时进行、输入是什么、相关工具版本、结果，以及证据位置。任务文件里写着一条计划运行的命令，不算验证记录。小型比较可能只需要简短复核说明；后果更重的工作，则可能需要测试日志、文件哈希、截图或独立复核。

自动检查擅长发现明确条件，例如文件缺失或状态页过期。它不一定能发现流畅文字中的错误解读。在我们的例子里，检查是否出现“无障碍”三个字，远弱于检查 Maple 的无障碍条件是否被正确标为未知。

区分文字指令与实际约束

一条指令可以要求 Agent 在预订前停下。受限的工具策略可以让它无法调用预订操作。校验器可以拒绝缺少证据的任务记录。运行环境的 hook 可以接收校验器结果并采取动作，但前提是接入已经被加载，而且实际行为符合预期。

这些机制相互配合，却不能互相替代。仓库的完成关卡 (closeout gate) 检查输出是否存在、验证状态是否已声明、记录路径是否有效、状态页是否最新、容量限制是否符合要求，以及任务锁是否释放。它不会独立判断报告中的每个结论是否真实。错误或不诚实的“passed”记录，仍然需要内容复核来发现。

运行环境也必须正确连接关卡。在终端里运行脚本并得到错误，不能证明宿主会阻止最终完成声明。这需要用故意不完整的任务，再用一个有效任务，在新会话中测试。静态证据与实际运行证据应分开保存。

让部分完成仍然有用

如果 Maple 的来源无法读取，就说明哪些内容检查过，哪些尚未核实。如果任务要求的是忠于证据的比较，那么“入口情况未知”可以是正确的最终结果。如果任务要求确认每个选项的无障碍条件，同一个缺口就会阻止完成。决定这一点的是任务约定。

无法继续时，保留可用文件，说明依赖什么，并写出下一步。适当时使用如实的失败或取消状态。不要为了通过关卡、或让看板显得整齐，编造成功记录。

好的验证让下一位使用者知道：可以依赖什么，应当检查什么，以及哪些新证据可能改变结论。[可靠性参考](#)进一步解释了任务、验证记录、完成关卡和恢复机制。

8. 维护真正用得上的积累

工作空间应当随着有用成果的积累，变得更容易继续使用。把所有内容都存下来，并不能自动达到这个目标。未经整理的聊天记录、重复流程、过期状态页和失效指令，反而可能让下一次会话更慢、更不可靠。

维护可以从一个实际问题开始：哪些保留的信息有助于未来任务，它们应放在哪里？保留决定、出处、有效方法和恢复线索。当材料所在的位置不再适合它的用途时，再移动、归档或退役。

把当前任务收好

场地比较结束后，保留具名来源、检查过的答案、相关验证说明和交接记录。没有必要把每个草稿都放进紧凑记忆。如果一次纠正揭示了可复用的经验，再判断它是否应补充到已有 Skill 或知识笔记中。

不是每个任务都需要修改记忆。记忆是简短的恢复辅助：当一个长期决定或线索否则难以重新找到时，它才有价值。当前进度留在任务里，大段解释放进知识或可引用的产物中。

使用标准项目骨架时，更新任务记录，再重新生成 STATUS.md，不要在多个位置分别维护进度。任务结束后，把最终结果和剩余限制写清楚，避免后来的读者把旧草稿误认成正在执行的任务。

根据真实摩擦做维护

维护频率应反映使用情况。经常使用的系统可以每周做一次简短检查；不常打开的项目，也许重新使用时检查即可。文档里写了一张时间表，只能算建议，直到实际配置并测试了调度器，才能称为已启用的自动任务。

寻找重复出现的问题：不断重讲背景、找不到来源、职责分配不清、启动文件过大、跳过验证、多个副本相互矛盾。修复造成问题的最小原因。一项 Skill 描述不清，可能只需要改一句话，不一定需要新 Agent 或新框架。

较大的调整可以留下简短的前后记录。例如：“新会话原来会读取整个归档；入口现在只指向当前任务，需要时再检索旧笔记。”然后在新会话中检查实际行为。只修改了指令，最多能证明文字发生变化。

控制启动时加载的内容

工具包为指令、记忆、任务记录和大文件索引设置了容量限制，目的是鼓励按需加载。它们并不意味着每个文件填满额度，效果就会更好。

文件过大时，先保护信息意义，再压缩长度。稳定解释移到知识，任务细节回到各自任务，已结束历史进入归档，启动位置保留简洁线索。不要为了通过大小检查，删除某个未解决决定的唯一记录。

大输入应有清单，记载路径、大小、用途和加载策略。模型可以先看索引，再读取相关部分，而不是一次读完整个资料库。受限材料仍然需要真正的访问控制；清单不是权限系统。

保持结构变化可恢复

合并 Agent、停用 Skill 或移动权威文件之前，先确认负责人、引用关系、活跃任务和恢复路径。适当保留可恢复版本，并为破坏性或外部动作取得所需授权。整齐的目录不能取代可理解的历史。

[系统复查与更新指南](#)提供更深入的做法。等积累足够真实使用经验后再展开评估。很多时候，最好的维护结果是系统更小了，而留下的每一部分都有清楚用途和实际价值。

9. 参考：使用标准工具包

首次项目目录演示的是一种小型工作方法。Portable Agentic System 工具包提供更完整的标准项目骨架，包含领域目录、任务约定、自动状态页、容量检查、锁和适配配置。当这些功能有用时，可以选择这条搭建路线；完成前面的学习练习并不要求先生成它。

下面的命令从本仓库根目录运行，使用本地 Python 脚本，前提是已经安装与源码兼容的 python3。执行前检查脚本、--help、目标目录和自己的环境。第一次生成时使用一个新的临时目录。本章说明如何调用，并不表示这些命令已在你的电脑上运行。

先预览，再生成

阅读[初始配置](#)；需要修改示例领域时，先复制到自己的工作文件。随仓库提供的配置含有两个示例负责人，并不是适合所有人的推荐。每个 Agent 都需要使命、路由描述、排除范围，以及能够区分相邻工作的例子。

选择尚未使用的目标路径，先预览：

```
python3 skills/portable-agentic-system/scripts/create_agentic_system.py \
--root /tmp/pas-learning-demo \
--config skills/portable-agentic-system/pas/examples/starter-config.json \
--dry-run
```

预览以 JSON 报告目标根目录与 Agent 路径，不会列出将写入的每个文件。创建前，结合这些路径检查模板和生成器实际生成结构的行为。需要实际创建这份已审阅的结构时，去掉同一命令中的 --dry-run 再运行。生成器默认拒绝与已有文件冲突；不要把 --force 当成常规报错修复，先查清冲突并保留已有工作。

生成的系统包含运行环境配置，以及 .pas/bin/ 下的本地运行脚本。在目标工具中信任或启用 hooks 前，应先审阅它们。生成器运行成功，不证明运行环境已经发现或执行这些配置。

运行静态检查

生成以后，下列命令用于检查结构。如果选择了其他路径，把目标一致地替换掉。

```
python3 skills/portable-agentic-system/scripts/validate_agentic_system.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/check_budgets.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/check_descriptions.py \
/tmp/pas-learning-demo
python3 skills/portable-agentic-system/scripts/generate_status.py \
/tmp/pas-learning-demo --check
python3 skills/portable-agentic-system/scripts/harness_health_check.py \
/tmp/pas-learning-demo --json
```

每项失败都应先读清原因。如果任务记录已经变化而状态页过期，使用同一 root 运行不带 --check 的 generate_status.py，再重新检查。重新生成会写入 STATUS.md，检查模式只核对一致性。

还可以运行适配探测：

```
python3 skills/portable-agentic-system/scripts/adaptor_smoke.py \
/tmp/pas-learning-demo --runtime codex
```

执行的仍是静态探测，尽管文件名中有 smoke。它报告的 verification_level 是 static。根据实际目标选择 runtime，并按对应适配文档进行新会话测试。配置检查通过，不能据此升级为运行时验证通过。

理解任务记录的权威

标准 tasks/<task-id>/task.yaml 记录任务标识与归属、目标与范围、允许读取和写入的内容、工具与禁止动作、输出和完成条件、验证记录、执行资源，以及交接。生成器在 .yaml 文件内使用 JSON 语法，便于标准库解析器读取。解析器也支持保守的 YAML 子集，并不承诺支持完整 YAML 语法。

手工构建前，先以生成的任务为格式示例，并查看[治理参考](#)。第 3 章的教学交接记录，并不是一份字段完整、可通过校验的任务清单。

对于已经写入生成系统的真实任务，可从该系统根目录调用随附的完成关卡：

```
python3 .pas/bin/closeout_gate.py . tasks/T-123/task.yaml
```

将 T-123 替换为已有任务。成功收尾需要声明的输出、验证状态与记录路径、最新的自动状态页、符合容量要求、已释放的任务锁，以及交接说明。失败或取消使用如实的非成功状态。关卡的结构检查不能替代内容复核，也不能证明宿主的阻止机制已经实际生效。

按需查阅深入说明

以下深层实现参考主要使用英文，精确标识符保持原文：

主题	维护中的参考
文件归属与加载	文件系统契约
私密资料与外部动作	隐私边界
路由与描述	路由评估
运行环境分类	兼容性清单
适配选择与检查	适配指南
任务、关卡和并发	可靠性参考
完整设计访谈	搭建工作簿

仓库的 Python 测试命令是 `python3 -m unittest discover -s tests -v`。通过测试能支持被测试的本地行为，但不能证明新会话兼容性、发布、送达或任何外部交易。每一种结果，都应按实际取得的证据分别报告。

延伸阅读

本版链接指向仓库及相关技术参考。产品接入说明有各自的证据日期；保留链接不代表本次已重新验证运行行为。

新手指南

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/docs/start-here.zh-CN.md>

快速开始

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/QUICKSTART.zh-CN.md>

Anthropic 对工作流与 Agent 的解释

<https://www.anthropic.com/engineering/building-effective-agents>

Agent Skills 规范

<https://agentskills.io/specification>

首次项目练习

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/examples/first-project/README.zh-CN.md>

文件系统契约

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/filesystem-contract.md>

expected/handoff.md

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/examples/first-project/expected/handoff.md>

Skill 模板

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/templates/skill/SKILL.md>

方法提炼指南

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/skill-distillation-and-fusion.md>

路由评估指南

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/description-routing-evals.md>

兼容性清单

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/compatibility/runtime-compatibility.json>

适配选择指南

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/adapters.md>

可靠性参考

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/textbook-reliability.md>

系统复查与更新指南

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/system-review-and-renewal.md>

初始配置

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/examples/starter-config.json>

治理参考

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/docs/reference/task-lifecycle.md>

隐私边界

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/privacy-boundaries.md>

搭建工作簿

<https://github.com/kwis7/Plug-And-Chug-Agentic-Building-Guide/blob/main/skills/portable-agentic-system/pas/references/textbook-build-workbook.md>